

请记住

- 请使用 `member function templates` (成员函数模板) 生成“可接受所有兼容类型”的函数。
- 如果你声明 `member templates` 用于“泛化 `copy` 构造”或“泛化 `assignment` 操作”，你还是需要声明正常的 `copy` 构造函数和 `copy assignment` 操作符。

条款 46：需要类型转换时请为模板定义非成员函数

Define non-member functions inside templates when type conversions are desired.

条款 24 讨论过为什么惟有 `non-member` 函数才有能力“在所有实参身上实施隐式类型转换”，该条款并以 `Rational class` 的 `operator*` 函数为例。我强烈建议你继续看下去之前先让自己熟稔那个例子，因为本条款首先以一个看似无害的改动扩充条款 24 的讨论；本条款将 `Rational` 和 `operator*` 模板化了：

```
template<typename T>
class Rational {
public:
    Rational(const T& numerator = 0,           //条款 20 告诉你为什么参数以
            const T& denominator = 1);        // passed by reference 方式传递。
    const T numerator() const;                 //条款 28 告诉你为什么返回值
    const T denominator() const;              // 以 passed by value 方式传递。
    ...                                        //条款 3 告诉你为什么它们是 const.
};

template<typename T>
const Rational<T> operator* (const Rational<T>& lhs,
                             const Rational<T>& rhs)
{ ... }
```

就像条款 24 一样，我们希望支持混合式 (`mixed-mode`) 算术运算，所以我们希望以下代码顺利通过编译。我们也预期它会，因为它正是条款 24 所列的同一份代码，唯一不同的是 `Rational` 和 `operator*` 如今都成了 `templates`：

```
Rational<int> oneHalf(1, 2);                  //这个例子来自条款 24，
                                              //唯一不同是 Rational 改为 template。
Rational<int> result = oneHalf * 2;           //错误！无法通过编译。
```

上述失败给我们的启示是，模板化的 `Rational` 内的某些东西似乎和其 `non-template` 版本不同。事实的确如此。在条款 24 内，编译器知道我们尝试调用什么函数（就是接受两个 `Rationals` 参数的那个 `operator*` 啦），但这里编译器不知道我们想要调用哪个函数。取而代之的是，它们试图想出什么函数被名为 `operator*`

的 `template` 具现化(产生)出来。它们知道它们应该可以具现化某个“名为 `operator*` 并接受两个 `Rational<T>` 参数”的函数, 但为完成这一具现化行动, 必须先算出 `T` 是什么。问题是它们没有这个能耐。

为了推导 `T`, 它们看了看 `operator*` 调用动作中的实参类型。本例中那些类型分别是 `Rational<int>` (`oneHalf` 的类型) 和 `int` (`2` 的类型)。每个参数分开考虑。

以 `oneHalf` 进行推导, 过程并不困难。`operator*` 的第一参数被声明为 `Rational <T>`, 而传递给 `operator*` 的第一实参 (`oneHalf`) 的类型是 `Rational<int>`, 所以 `T` 一定是 `int`。其他参数的推导则没有这么顺利。`operator*` 的第二参数被声明为 `Rational<T>`, 但传递给 `operator*` 的第二实参 (`2`) 类型是 `int`。编译器如何根据这个推算出 `T`? 你或许会期盼编译器使用 `Rational<int>` 的 `non-explicit` 构造函数将 `2` 转换为 `Rational<int>`, 进而将 `T` 推导为 `int`, 但它们不这么做, 因为在 `template` 实参推导过程中从不将隐式类型转换函数纳入考虑。绝不! 这样的转换在函数调用过程中的确被使用, 但在能够调用一个函数之前, 首先必须知道那个函数存在。而为了知道它, 必须先为相关的 `function template` 推导出参数类型 (然后才可将适当的函数具现化出来)。然而 `template` 实参推导过程中并不考虑采纳“通过构造函数而发生的”隐式类型转换。条款 24 不涉及 `templates`, 所以 `template` 实参推导不成为讨论议题。现在我们却是处在 `template part of C++` (见条款 1) 领域内, `template` 实参推导是我们的重大议题。

只要利用一个事实, 我们就可以缓和编译器在 `template` 实参推导方面受到的挑战: `template class` 内的 `friend` 声明式可以指涉某个特定函数。那意味 `class Rational<T>` 可以声明 `operator*` 是它的一个 `friend` 函数。`Class templates` 并不倚赖 `template` 实参推导 (后者只施行于 `function templates` 身上), 所以编译器总是能够在 `class Rational<T>` 具现化时得知 `T`。因此, 令 `Rational<T>` `class` 声明适当的 `operator*` 为其 `friend` 函数, 可简化整个问题:

```
template<typename T>
class Rational {
public:
    ...
```

```

friend .                                     //声明
const Rational operator*(const Rational& lhs, //operator* 函数,
                        const Rational& rhs); //细节详下。

};

template<typename T>                         //定义
const Rational<T> operator*(const Rational<T>& lhs, //operator* 函数。
                        const Rational<T>& rhs)
{ ... }

```

现在对 `operator*` 的混合式调用可以通过编译了，因为当对象 `oneHalf` 被声明为一个 `Rational<int>`，`class Rational<int>` 于是被具现化出来，而作为过程的一部分，`friend` 函数 `operator*`（接受 `Rational<int>` 参数）也就被自动声明出来。后者身为一个函数而非函数模板（`function template`），因此编译器可在调用它时使用隐式转换函数（例如 `Rational` 的 `non-explicit` 构造函数），而这便是混合式调用之所以成功的原因。

但是，此情境下的“成功”是个有趣的字眼，因为虽然这段代码通过编译，却无法连接。稍后我马上回来处理这个问题，首先我要谈谈在 `Rational` 内声明 `operator*` 的语法。

在一个 `class template` 内，`template` 名称可被用来作为“`template` 和其参数”的简略表达方式，所以在 `Rational<T>` 内我们可以只写 `Rational` 而不必写 `Rational<T>`。本例中这只节省我们少打几个字，但若出现许多参数，或参数名称很长，这可以节省我们的时间，也可以让代码比较干净。我谈这个是因为，本例中的 `operator*` 被声明为接受并返回 `Rationals`（而非 `Rational<T>s`）。如果它被声明如下，一样有效：

```

template<typename T>
class Rational {
public:
    ...
    friend
        const Rational<T> operator*(const Rational<T>& lhs,
                                    const Rational<T>& rhs);
    ...
};

```

然而使用简略表达方式（速记式）比较轻松也比较普遍。

现在回头想想我们的问题。混合式代码通过了编译，因为编译器知道我们要调用哪个函数（就是接受一个 `Rational<int>` 以及又一个 `Rational<int>` 的那个

operator*)，但那个函数只被声明于 Rational 内，并没有被定义出来。我们意图令此 class 外部的 operator* template 提供定义式，但是行不通——如果我们自己声明了一个函数（那正是 Rational template 内的作为），就有责任定义那个函数。既然我们没有提供定义式，连接器当然找不到它！

或许最简单的可行办法就是将 operator* 函数本体合并至其声明式内：

```
template<typename T>
class Rational {
public:
...
friend const Rational operator*(const Rational& lhs,
                                const Rational& rhs)
{
    return Rational(lhs.numerator() * rhs.numerator(),    //实现码与
                    lhs.denominator() * rhs.denominator()); //条款 24 同
}
};
```

这便如同我们所期望地正常运作了起来：对 operator* 的混合式调用现在可编译连接并执行。万岁！

这项技术的一个趣味点是，我们虽然使用 friend，却与 friend 的传统用途“访问 class 的 non-public 成分”毫不相干。为了让类型转换可能发生于所有实参身上，我们需要一个 non-member 函数（条款 24）；为了令这个函数被自动具现化，我们需要将它声明在 class 内部；而在 class 内部声明 non-member 函数的唯一办法就是：令它成为一个 friend。因此我们就这样做了。不习惯？是的。有效吗？不必怀疑。

一如条款 30 所说，定义于 class 内的函数都暗自成为 inline，包括像 operator* 这样的 friend 函数。你可以将这样的 inline 声明所带来的冲击最小化，做法是令 operator* 不做任何事情，只调用一个定义于 class 外部的辅助函数。在本条款的例子中，这样做并没有太大意义，因为 operator* 已经是个单行函数，但对更复杂的函数而言，那么做也许就有价值。“令 friend 函数调用辅助函数”的做法的确值得细究一番。

“Rational 是个 template”这一事实意味上述的辅助函数通常也是个 template，所以定义了 Rational 的头文件代码，很典型地长这个样子：

```
template<typename T> class Rational;    //声明 Rational template
```

```

template<typename T>                                //声明 helper template
const Rational<T> doMultiply(const Rational<T>& lhs,
                             const Rational<T>& rhs);

template<typename T>
class Rational {
public:
    ...
friend
    const Rational<T> operator*(const Rational<T>& lhs,
                                const Rational<T>& rhs)
    { return doMultiply(lhs, rhs); }                //令 friend 调用 helper
    ...
};

```

许多编译器实质上会强迫你把所有 `template` 定义式放进头文件内，所以你或许需要在头文件内定义 `doMultiply`（一如条款 30 所言，这样的 `templates` 不需非得是 `inline` 不可），看起来像这样：

```

template<typename T>                                //若有必要，
const Rational<T> doMultiply(const Rational<T>& lhs, //在头文件内定义
                             const Rational<T>& rhs) //helper template
{
    return Rational<T>(lhs.numerator() * rhs.numerator(),
                       lhs.denominator() * rhs.denominator());
}

```

作为一个 `template`，`doMultiply` 当然不支持混合式乘法，但它其实也不需要。它只被 `operator*` 调用，而 `operator*` 支持了混合式操作！本质上 `operator*` 支持了类型转换所需的任何东西，确保两个 `Rational` 对象能够相乘，然后它将这两个对象传给一个适当的 `doMultiply` `template` 具现体，完成实际的乘法操作。协作为成功之本，不是吗？

请记住

- 当我们编写一个 `class template`，而它所提供之“与此 `template` 相关的”函数支持“所有参数之隐式类型转换”时，请将那些函数定义为“`class template` 内部的 `friend` 函数”。

条款 47：请使用 traits classes 表现类型信息

Use traits classes for information about types.

STL 主要由“用以表现容器、迭代器和算法”的 `templates` 构成，但也覆盖若干工具性 `templates`，其中一个名为 `advance`，用来将某个迭代器移动某个给定距离：

```
template<typename IterT, typename DistT> //将迭代器向前移动 d 单位。  
void advance(IterT& iter, DistT d); //如果 d < 0 则向后移动。
```

观念上 `advance` 只是做 `iter += d` 动作, 但其实不可以全然那么实践, 因为只有 **random access** (随机访问) 迭代器才支持 `+=` 操作。面对其他威力不那么强大的迭代器种类, `advance` 必须反复施行 `++` 或 `--`, 共 `d` 次。

嗯, 你不记得你的 STL 迭代器分类 (categories) 了吗? 没关系, 让我们来一次迷你回顾。STL 共有 5 种迭代器分类, 对应于它们支持的操作。**Input** 迭代器只能向前移动, 一次一步, 客户只可读取 (不能涂写) 它们所指的东 西, 而且只能读取一次。它们模仿指向输入文件的阅读指针 (read pointer); C++ 程序库中的 `istream_iterators` 是这一分类的代表。**Output** 迭代器情况类似, 但一切只为输出: 它们只向前移动, 一次一步, 客户只可涂写它们所指的东 西, 而且只能涂写一次。它们模仿指向输出文件的涂写指针 (write pointer); `ostream_iterators` 是这一分类的代表。这是威力最小的两个迭代器分类。由于这两类都只能向前移动, 而且只能读或写其所指物最多一次, 所以它们只适合“一次性操作算法” (one-pass algorithms)。

另一个威力比较强大的分类是 **forward** 迭代器。这种迭代器可以做前述两种分类所能做的每一件事, 而且可以读或写其所指物一次以上。这使得它们可施行于多次性操作算法 (multi-pass algorithms)。STL 并未提供单向 linked list, 但某些程序库有 (通常名为 `slist`), 而指入这种容器的迭代器就是属于 **forward** 迭代器。指入 TR1 **hashed** 容器 (见条款 54) 的也可能是这一分类。(译注: 这里说“可能”是因为 **hashed** 容器的迭代器可为单向也可为双向, 取决于实现版本。)

Bidirectional 迭代器比上一个分类威力更大: 它除了可以向前移动, 还可以向后移动。STL 的 `list` 迭代器就属于这一分类, `set`, `multiset`, `map` 和 `multimap` 的迭代器也都是这一分类。

最有威力的迭代器当属 **random access** 迭代器。这种迭代器比上一个分类威力更大的地方在于它可以执行“迭代器算术”, 也就是它可以在常量时间内向前或向后跳跃任意距离。这样的算术很类似指针算术, 那并不令人惊讶, 因为 **random access** 迭代器正是以内置 (原始) 指针为榜样, 而内置指针也可被当做 **random access** 迭代器使用。`vector`, `deque` 和 `string` 提供的迭代器都是这一分类。

对于这 5 种分类, C++ 标准程序库分别提供专属的卷标结构 (tag struct) 加以确认: